

Lab 0 实验报告

使用 OpenROAD 自动完成芯片设计

Tongbin Zhao

2026.9.15

本报告回答实验指导书 Questions 中的问题 1–5。实验对象为 ORFS 内置的 `designs/sky130hd/jpeg`, 顶层模块为 `jpeg_encoder`, 基线时钟周期为 `5.0ns` (`200MHz`)。日志记录的 OpenROAD 版本为 `26Q3-2056-g41a28926b9`, 标准单元库为 `sky130_fd_sc_hd_tt_025C_1v80.lib`, 即 `TT`、`25°C`、`1.80V` 条件。

下文文件路径均相对于 `OpenROAD-flow-scripts/flow/`。记 $L = \text{logs/sky130hd/jpeg/base/}$, $R = \text{reports/sky130hd/jpeg/base/}$, $D = \text{results/sky130hd/jpeg/base/}$; 扫描结果目录记为 $S = \text{tmp/clock_period_sweep_sky130hd_jpeg_jmed41fq/}$ 。数值以这些目录中的已有文件为依据。

1 问题 1: OpenROAD 运行的各个阶段

按 L 中实际存在的 21 个 `.log` 文件归类, OpenROAD 流程包括综合、布图规划、布局、时钟树综合、布线和收尾六个主阶段。表中的次阶段名即日志文件名去掉 `.log` 后的名称。两个 LEC 日志是所属主流程内部调用的等价性检查。

表 1: 按日志命名划分的流程及各次阶段作用

主阶段	次阶段 (日志名)	主要作用
1 综合	<code>1_1_yosys_canonicalize</code>	读取 RTL、展开参数化模块并整理设计层次, 生成供后续综合使用的 RTLIL 中间表示。
	<code>1_2_yosys</code>	将行为级设计转换、优化并映射到 sky130hd 标准单元, 输出门级网表和约束。
	<code>1_synth</code>	在 OpenROAD 中载入综合网表、工艺库和约束, 生成综合后的 ODB 数据库并统计设计规模。
2 布图规划	<code>2_1_floorplan</code>	建立 die、core、标准单元行与布线轨道, 调整 tie 单元及缓冲器, 并生成初始时序报告。
	<code>2_2_floorplan_macro</code>	为宏单元安排位置; 本设计没有宏单元, 日志明确记录跳过宏布局。
	<code>2_3_floorplan_tapcell</code>	插入 10,174 个 tapcell, 为阱和衬底提供连接。
	<code>2_4_floorplan_pdn</code>	生成芯片电源分配网络 PDN, 建立标准单元区域的供电结构。
3 布局	<code>3_1_place_gp_skip_io</code>	在暂不考虑 I/O 引脚位置的条件下执行初步全局布局。

表 1 (续)

主阶段	次阶段 (日志名)	主要作用
	3_2_place_iop	根据初步布局优化边界 I/O 引脚位置, 以改善引脚到内部单元的连线。
	3_3_place_gp	执行时序和可布线性驱动的全局布局, 并插入端口缓冲器及进行设计修复。
	3_4_place_resized	基于布局估计寄生参数, 检查并修复单元驱动能力和电气设计约束。
	3_5_place_dp	将单元合法化到具体 site 和行, 优化镜像方向, 并检查布局合法性和时序。
4 时钟树综合	4_1_cts	为 4,384 个时钟接收端构建时钟树, 并在真实时钟延迟下修复 setup/hold 时序。
	4_rsz_lec_check	检查 CTS 后时序修复前后的逻辑等价性, 本次结果为 Circuits are IDENTICAL。
5 布线	5_1_grt	执行全局布线、生成布线引导, 并进行寄生估计、时序修复与天线修复。
	5_2_route	完成精确轨道和过孔上的详细布线, 迭代消除布线规则违例, 并继续修复天线问题。
	5_3_fillcell	在标准单元间隙放置 60,829 个 filler, 补齐单元行的物理连续性。
6 收尾	6_1_fill	执行金属填充阶段入口; 本次实际操作为复制布线数据库, 日志没有金属填充插入记录。
	6_report	完成最终电源连接、网表与 DEF 输出、RC 提取、STA、IR drop 分析和结果图像生成。
	6_final_lec_check	比较综合后的逻辑网表与最终逻辑网表, 本次等价性检查通过。
	6_1_merge	用 KLayout 将最终 DEF 与单元版图合并为 GDS, 检查单元版图匹配及孤立单元。

收尾阶段按实际依赖关系列出: 6_final_lec_check 在 6_report 内部执行, 6_1_merge 使用最终 DEF 生成版图。本次 6_1_merge.log 报告所有 LEF 单元都有匹配的 GDS/OAS 单元, 且最终版图没有 orphan cell; 最终产物为 D 下的 6_final.gds。

2 问题 2: 设计规模、时序和 DRC

本题中的实力数量统计数据来源于 `OpenROAD-flow-scripts/flow/reports/sky130hd/jpeg/base/synth_stat.txt` (以下简称 `stat.txt`)。、“总实例”包含逻辑单元和实际插入的物理单元, 在 `stat.txt` 中对应 cell 一栏数据; “时序实例”指寄存器/触发器, 在 `stat.txt` 中对应 `sky130_fd_sc_hd__edfxtpt*` 和 `sky130_fd_sc_hd__dftrtp*` 这两类数据的 count 之和; “组合实例”包含其余所有标准单元项目, 即逻辑门、反相器、缓冲器和常量单元。综合统计列出 $4,318 + 64 + 2 = 4,384$ 个时序单元。表中上标 †

表示依据相邻已知网表/统计及该次阶段的单元增删记录推算或沿用，并非该阶段独立输出的分类统计。无独立时序报告的次阶段标记为“—”。

表中 WNS/TNS 均为 setup 指标，单位为 ns；优先采用同名 JSON 中的 `timing__setup__ws` 和 `timing__setup__tns`。由于按照 `lab0_guide.pdf` 要求用默认的时钟周期 5ns 跑完整的 flow，时序优化的最终结果依然无法满足 setup 时序约束，因此本表中已记录的 worst setup slack 全为负数。

DRC 列专指**详细布线规则违例数**，未计入布局合法性、天线违例、max slew/max capacitance 检查或工具 `errors__count`，数据来源为 `OpenROAD-flow-scripts/flow/logs/sky130hd/jpeg/base/5_2_route.json`。`OpenROAD-flow-scripts/flow/reports/sky130hd/jpeg/base/5_route_drc.rpt` 为空，因此可以判定最后 drc 为例数目为 0。

表 2: 各主、次阶段的设计规模与质量指标（基线 $T = 5.0\text{ ns}$ ）

次阶段	总实例	时序实例	组合实例	WNS/ns	TNS/ns	DRC
1 综合						
1_1_yosys_canonicalize	—	—	—	—	—	—
1_2_yosys	31,722	4,384	27,338	—	—	—
1_synth	31,722	4,384 [†]	27,338 [†]	—	—	—
2 版图规划						
2_1_floorplan	30,811	4,384 [†]	26,427 [†]	-49.6288	-58,959.2	—
2_2_floorplan_macro	30,811 [†]	4,384 [†]	26,427 [†]	—	—	—
2_3_floorplan_tapcell	40,985 [†]	4,384 [†]	26,427 [†]	—	—	—
2_4_floorplan_pdn	40,985 [†]	4,384 [†]	26,427 [†]	—	—	—
3 布局						
3_1_place_gp_skip_io	40,985 [†]	4,384 [†]	26,427 [†]	—	—	—
3_2_place_iop	40,985 [†]	4,384 [†]	26,427 [†]	—	—	—
3_3_place_gp	45,041	4,384 [†]	30,483 [†]	-0.871954	-149.194	—
3_4_place_resized	45,041	4,384 [†]	30,483 [†]	-0.871954	-149.194	—
3_5_place_dp	45,041	4,384 [†]	30,483 [†]	-0.844211	-152.148	—
4 时钟树综合						
4_1_cts	46,045	4,384	31,487 [†]	-0.663021	-124.882	—
4_rsz_lec_check	—	—	—	—	—	—
5 布线						
5_1_grt	46,155	4,384 [†]	31,554 [†]	-0.780836	-166.273	—
5_2_route	46,210 [†]	4,384 [†]	31,554 [†]	—	—	0
5_3_fillcell	107,039 [†]	4,384 [†]	31,554 [†]	—	—	0*
6 收尾						
6_1_fill	107,039 [†]	4,384 [†]	31,554 [†]	—	—	0*
6_report	107,039	4,384	31,554	-0.610091	-87.6604	0*
6_final_lec_check	—	—	—	—	—	—

表 2 (续)

次阶段	总实例	时序实例	组合实例	WNS/ns	TNS/ns	DRC
6_1_merge	107,039 [†]	4,384 [†]	31,554 [†]	—	—	0 [*]

布线 DRC 的数据来自 5_2_route.json 的 detailedroute__route__drc_errors=0。上标 * 表示沿用这一检查结果，因为 filler、收尾或 GDS 合并阶段没有重新执行 DRC。现有文件没有独立 signoff DRC 结果。

3 问题 3：时钟周期扫描与最大频率

3.1 实验脚本

本实验通过脚本 scripts/sta_clock_period_sweep.py 进行。[点击跳转到报告末尾的完整实验脚本 \(附录 B\)](#)。脚本以已经完成布线的基线为输入，流程如下：

1. 从 *D* 读取 6_final.odb (最终物理设计及连接)、6_final.sdc (基线时钟和 I/O 等约束) 与 6_final.spef (已提取的布线寄生参数)。通过原设计 config.mk 和 load.tcl 载入相同的工艺/时序库；从 *L* 的 5_2_route.json 读取并保存基线 DRC 数。
2. 对每个扫描周期产生独立的 scan.sdc，同步修改 clk 和虚拟时钟 vclk_clk 的 create_clock -period。两者基线周期均为 5 ns。输入/输出延迟仍保持 1 ns，虚拟时钟延迟仍保持 1.175 ns，其余约束不变。脚本将 ns 转换为秒，再通过 sta::time_sta_ui 转成库的原生时间单位。
3. 在 tmp 中为每个周期建立独立目录，链接已有 ODB/SPEF，用 make run 执行临时生成的 sta.tcl。Tcl 调用 load_design 6_final.odb scan.sdc，设置 propagated clock 并读取已有 SPEF，随后在 OpenROAD 内重跑 OpenSTA。因此重分析对象是**布线完成后的最终设计状态**，对应收尾阶段的最终 STA；此过程不执行综合、布局、CTS、布线或重新提取寄生。
4. 输出 worst setup slack、WNS、TNS，以及各路径组至多 5 条 setup/hold 路径；解析 sta.log，生成 summary.csv 和 summary.txt。setup 是否通过由未经低精度取整的 worst setup slack 是否非负判定。

脚本保留 generated-clock 关系；对显式 waveform 或基线周期不一致的主/虚拟时钟会拒绝扫描，以免在未经核实时改变时钟语义。它虽输出 hold 路径报告，但汇总中的 setup_pass 只表示本题要求的 setup 判据。

S 的 experiment.json 记录了本次实验的时钟频率范围为 5.60–5.70 ns、步长 0.01 ns，共 11 点。这与脚本默认步长 0.1 ns 不同；与记录一致的运行命令为：

```
cd OpenROAD-flow-scripts/flow
python3 scripts/sta_clock_period_sweep.py \
  --platform sky130hd --design jpeg --min 5.6 --max 5.7 --step 0.01
```

3.2 扫描结果

表 3 的基线来自 6_report.json, 11 个扫描点来自 S 的 summary.csv, 并与各周期的 sta.log 核对。正的 worst setup slack 与 WNS 不同: OpenSTA 的 report_wns 在没有负 slack 时返回 0。因此同时列出两者, 避免把通过时的裕量误写为 WNS。

表 3: 固定最终布线设计的时钟周期扫描

	T/ns	f/MHz	最差 setup slack/ns	WNS/ns	TNS/ns	DRC	setup
5.00 (基线)	200.000000		-0.610091	-0.610091	-87.6604	0	失败
5.60	178.571429		-0.010091039	-0.010091039	-0.010091039	0	失败
5.61	178.253119		-0.000091038	-0.000091038	-0.000091038	0	失败
5.62	177.935943		0.009908963	0.000000000	0.000000000	0	通过
5.63	177.619893		0.019908965	0.000000000	0.000000000	0	通过
5.64	177.304965		0.029908964	0.000000000	0.000000000	0	通过
5.65	176.991150		0.039908968	0.000000000	0.000000000	0	通过
5.66	176.678445		0.049908966	0.000000000	0.000000000	0	通过
5.67	176.366843		0.059908967	0.000000000	0.000000000	0	通过
5.68	176.056338		0.069908969	0.000000000	0.000000000	0	通过
5.69	175.746924		0.079908967	0.000000000	0.000000000	0	通过
5.70	175.438596		0.089908972	0.000000000	0.000000000	0	通过

基线的 setup 裕量为负, 由题目给出的关系, 可先估计限制周期

$$T_{\min, \text{est}} = T - s_{\text{worst}} = 5.0 - (-0.610091) \approx 5.610091 \text{ ns}.$$

其中 s_{worst} 使用带符号的 worst setup slack。扫描在 5.61 ns 时仍有 -0.000091038 ns (约 -0.091038 ps) 的负裕量, 即使低精度报告可能显示为 -0.00 , 仍应判为失败。5.62 ns 时裕量为 $+0.009908963$ ns, WNS 和 TNS 都为 0。

因此, 本次扫描中最小的通过周期为 5.62 ns, 对应最大已验证频率

$$f_{\max, \text{tested}} = \frac{1000}{5.62} = 177.935943 \text{ MHz}, \quad N_{\text{DRC}} = 0.$$

0.01 ns 的扫描步长只能把 setup 边界夹在 (5.61, 5.62] ns 内, 不能证明精确最小周期恰为 5.62 ns。由各点 $T - s_{\text{worst}}$ 得到的边界估计约为 5.610091 ns, 对应约 178.250227 MHz; 这也是最终低精度 period_min=5.61、fmax=178.25 报告的来源, 属于估计值, 未在该精确周期上实测通过。各点 DRC 都为 0, 是因为固定版图并沿用同一次详细布线检查结果, 而不是逐点重新执行 DRC。

4 问题 4: 运行时间占比与瓶颈

在基线 21 个日志中, 19 个有 Elapsed time, 可以直接提取墙钟时间。两个 LEC 日志没有独立的 Elapsed time, 但它们由 4_1_cts 和 6_report 内部同步调用, 耗时已包含在相应父阶段中, 故不重复

累加。其日志首末时间戳分别相差约 2.749 s 和 7.042 s，仅用于交叉核对。采用每个主阶段所含非重复次阶段的墙钟时间之和 t_i ，计算

$$\text{share}_i = \frac{t_i}{\sum_j t_j} \times 100\%, \quad \sum_j t_j = 1654.24 \text{ s} = 27 \text{ min } 34.24 \text{ s}.$$

表 4: 六个主阶段的运行时间及占比

主阶段	墙钟时间/s	占比/%
1 综合	36.45	2.20
2 布图规划	4.82	0.29
3 布局	76.48	4.62
4 时钟树综合	63.24	3.82
5 布线	1440.26	87.06
6 收尾	32.99	1.99
总计	1654.24	100.00

表 5: 次阶段耗时明细；占比的分母仍为整个基线流程的 1654.24 s

主阶段	次阶段	时间/s	占比/%
1 综合	1_1_yosys_canonicalize	1.88	0.11
	1_2_yosys	33.57	2.03
	1_synth	1.00	0.06
2 布图规划	2_1_floorplan	3.10	0.19
	2_2_floorplan_macro	0.39	0.02
	2_3_floorplan_tapcell	0.40	0.02
	2_4_floorplan_pdn	0.93	0.06
3 布局	3_1_place_gp_skip_io	3.96	0.24
	3_2_place_iop	0.41	0.02
	3_3_place_gp	47.03	2.84
	3_4_place_resized	8.61	0.52
	3_5_place_dp	16.47	1.00
4 时钟树综合	4_1_cts	63.24	3.82
	4_rsz_lec_check	已包含于父阶段，不另计	
5 布线	5_1_grt	252.87	15.29
	5_2_route	1186.35	71.72
	5_3_fillcell	1.04	0.06

表 5 (续)

主阶段	次阶段	时间/s	占比/%
6 收尾	6_1_fill	0.59	0.04
	6_report	29.30	1.77
	6_final_lec_check	已包含于父阶段, 不另计	
	6_1_merge	3.10	0.19

耗时的主要瓶颈是详细布线。单独的 5_2_route 用时 1186.35 s, 约占总耗时的 71.72%; 连同全局布线, 布线主阶段占 87.06%。详细布线需要为大量引脚寻找可达接入点, 并在具体轨道和过孔上反复搜索、检查规则和修复冲突, 因此程序运行耗时较长。

5 问题 5: ERROR、WARNING/WARN 及 waive 判断

按照题干, 对 L 下全部 21 个 .log 和 S 下 22 个 sta.log/make.log 进行不区分大小写的 ERROR、WARNING、WARN 检索; R 下两个天线检查日志也已检索, 未发现这些等级的消息。**没有发现 ERROR 级别报错**; 基线共 127 条警告消息, 扫描日志中 22 条均为 STA-0450, 其中 make.log 和 sta.log 是同一次运行的重复记录。不能为了填表而将 WARNING 改写为 ERROR。下表保留实际级别, 并将同原因的重复消息合并。waive 指是否可在本实验目的下接受该消息, 不等于证明设计通过完整签核。

表 6: 实际日志中的报错/警告原因与处置判断

主阶段 / 次阶段	原始消息与原因	是否可 waive
全流程与扫描 STA	未检出 ERROR	—
1 综合 / 1_1_yosys_canonicalize	Warning: Replacing memory ... with list of registers, 共 4 条: sresult、q_pipe、s_pipe、d_pipe 以寄存器形式展开; 见第 32、36–38 行。	可以。该实现使用标准单元寄存器, 综合检查为 0 problems; 应接受其面积代价, 不能将这些数组当成已映射的 SRAM。
1 综合 / 1_1_yosys_canonicalize	Warning: Ignoring module ... because it contains processes (run 'proc' command first), 共 74 条; 第 117–190 行发生在早期 OPT_CLEAN, 此时模块仍含行为过程。	可以在本次流程中接受。跳过的是该次清理, 后续 1_2_yosys 执行过程转换和映射; 最终综合统计 processes 为 0, synth_check.txt 报告 0 problems。
1 综合 / 1_synth; 4 CTS / 4_1_cts; 6 收尾 / 6_report	Warning: master sky130_ef_sc_hd__decap_12 has signal pins but no liberty cell, 共 5 条; 对应第 17 行、110/457 行、14/15 行, LEF 中的 decap master 没有匹配的 Liberty 单元。	本次可以。2_1_floorplan.log 明确说明该 master 已载入但未使用; 最终实例也未使用它, 故不形成待分析的逻辑时序弧。

表 6 (续)

主阶段 / 次阶段	原始消息与原因	是否可 waive
2 版图规划 / 2_1_floorplan	WARNING IFP-0028 (第 18 行): core 左下角从 (1.000,1.000) 对齐到 (1.380,2.720) μm , 以适配 site/row 网格。	可以。属于合法的网格对齐, 后续应以实际 core 坐标和面积为准。
2 版图规划 / 2_4_floorplan_pdn	WARNING PDN-1051 (第 10-11 行): CORE_macro_grid_1/2 没有对应实例。	可以。本设计宏单元数为 0, 2_2_floorplan_macro 已记录无宏; 标准单元电网仍正常生成。
3 布局 / 3_3_place_gp	WARNING GRT-0281 (第 157 行): Net clk has a large fanout of 4385 terminals, CTS 之前时钟尚直接连接大量触发器。	可以作为 CTS 前的阶段性警告接受。后续 CTS 对 4,384 个 sink 建树; 不能把 CTS 前的原始扇出当成最终时钟网的扇出。
4 CTS / 4_1_cts; 5 布线 / 5_1_grt	WARNING RSZ-0062: Unable to repair all setup violations (第 452、589 行); 修复后仍存在 setup 违例。	不能作为 5 ns 时序通过而 waive。 最终 WNS 仍为负; 只能将基线作为未收敛结果如实保留, 并在问题 3 中另行报告放宽周期后的 setup 通过结果。
5 布线 / 5_1_grt、 5_2_route	WARNING DRT-0349: LEF58_ENCLOSURE with no CUTCLASS is not supported, 各 10 条; 分别见第 12-21、13-22 行, 工具跳过 mcon、via 至 via4 的有关规则。	不能据此宣称完整 DRC 通过。 若用于制造签核, 须用支持这些规则的 DRC 检查补证, 现有结果不足以无条件 waive。
5 布线 / 5_1_grt、 5_2_route、5_3_fillcell; 6 收尾 / 6_report; 扫描 / 各 period_*/sta.log	WARNING STA-0450: virtual clock vclk_clk can not be propagated; 基线 4 条, 扫描每点 1 条 (另有 make 日志副本); set_propagated_clock [all_clocks] 同时选中了虚拟时钟。	可以。虚拟时钟没有实际时钟树, 本应保持理想时钟; 真实 clk 仍按传播时钟分析, 虚拟时钟的 I/O 约束仍保留。
5 布线 / 5_1_grt	WARNING RSZ-0104: Net ... only has one pin, 第 170-196 行共 12 个单引脚网络; 可能为悬空或未使用连接, 需核对设计意图。	后续 LEC 通过提供逻辑等价性的支持, 在认可 jpeg 这个设计没有问题的情况下可以 waive, 更严谨的做法是检查 jpeg 设计中的端口连接是否符合设计意图。
5 布线 / 5_1_grt	WARNING FLW-0010: more than one clock found. Skipping sdc update (第 686 行); 脚本发现 clk 和 vclk_clk, 跳过只支持单时钟的参考 SDC 更新。	本次可以。原有两时钟约束被保留, 6_final.sdc 中两者均为 5 ns; 问题 3 的脚本又同步修改了两者。不能理解为自动更新已经执行。
6 收尾 / 6_report	WARNING GUI-0076 (第 69 行): XDG_RUNTIME_DIR 未设置, Qt 使用 /tmp/runtime- 作为默认目录。	可以。属于 GUI 运行目录提示, 结果图像和最终产物已生成, 不是设计时序或物理违例。

6 问题 6：版图与关键路径

完成 5ns 时钟周期的完整 flow 运行后，在 OpenROAD-flow-scripts/flow 目录下运行

```
make gui_final
```

即可打开 OpenROAD 的 GUI 界面，根据用户导引中的相关指南打开 Heatmap-RoutingCongestion 图层，显示结果如下：

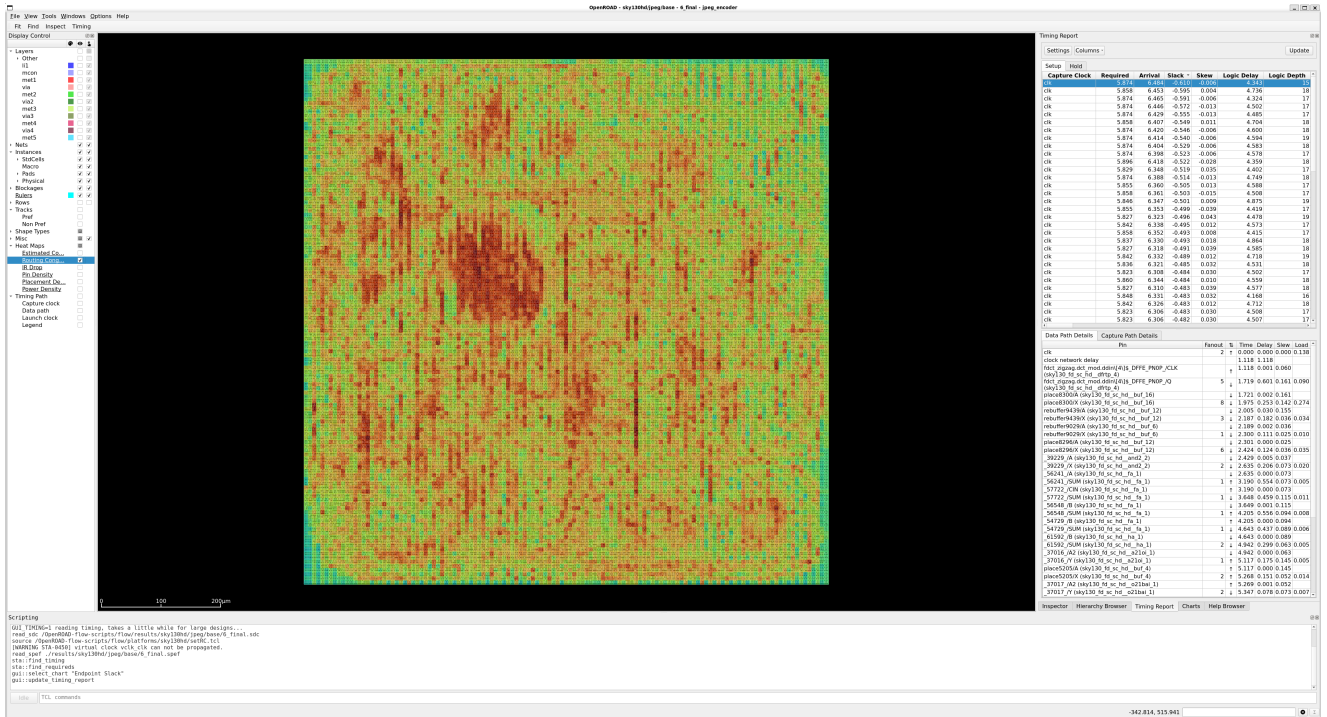


图 1: 5 ns 基线最终版图的布线拥塞热力图

在右侧的 Timing Report 菜单栏中点击选中 WNS 数值最小的一项，此条即本设计中的关键路径，可在 GUI 中显示出来。为了关键路径清晰可见此处关掉了 Layers 图层。

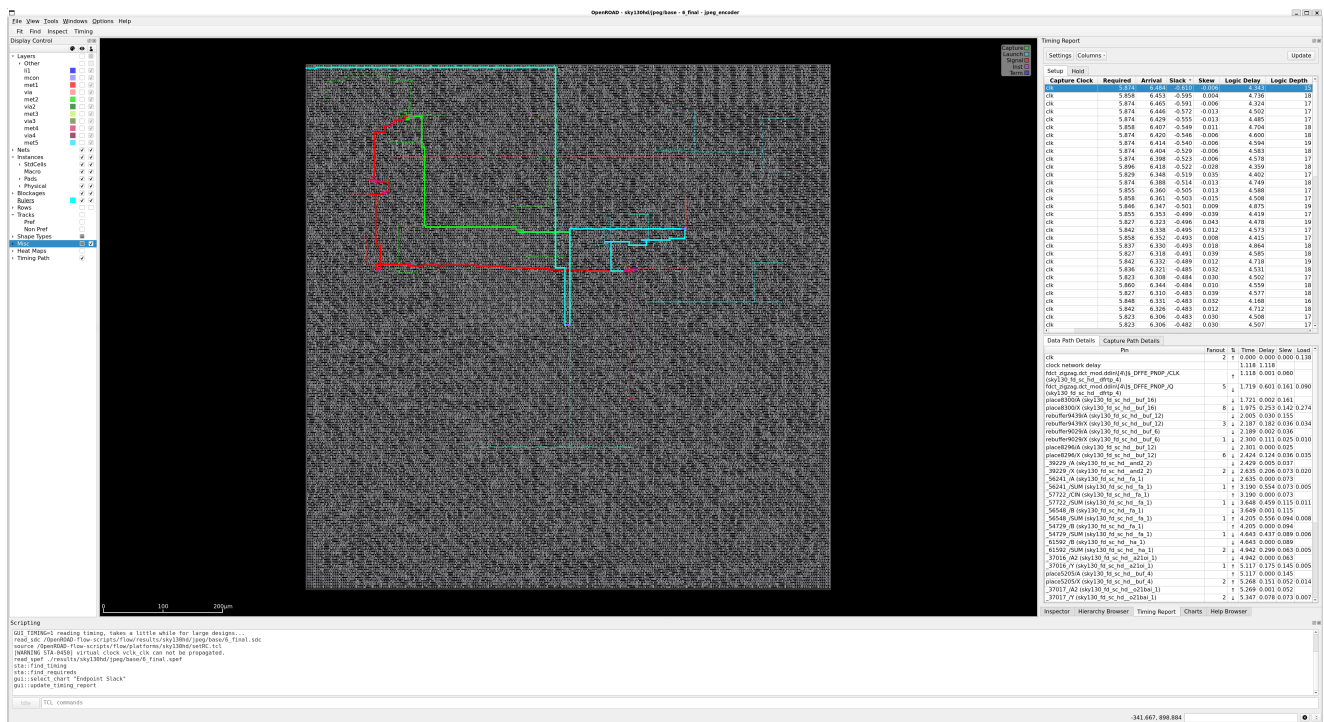


图 2: 5 ns 基线最终版图的关键时序路径

A AI 使用记录

本次 lab0 完成过程中，作者主要在 OpenSTA 重跑扫描时钟频率、基于 OpenROAD 输出结果的统计工作、报告撰写和实验环境配置过程中的 debug 四个方面使用了 AI 工具，其中后两项主要工程性的工作，不属于 lab0 的核心内容，因此此处不作详细展示。前两项中使用的 Prompt 分别如下：

A.1 OpenSTA 重跑扫描时钟频率

1. 如何只重跑 opensta?
2. AutoTuner 运行扫描的时候会不会重跑整个 flow 吗？还是如果只有 SDC_CLOCK_PERIOD 参数的话它就是只跑最后的 OpenSTA 的？
3. 请帮我完善一下前面的这个脚本，具体包含下面的步骤：
 - (a) 规定 clk_period 扫描实验的所有输出放在 ./tmp 目录下，我会在容器里面的 /OpenROAD-flow-scripts/flow 执行实验启动的命令
 - (b) 扫描的参数为 clk_period，扫描范围从 5.6 到 5.7 ns，间隔为 0.1ns
 - (c) 目前我已经跑完了 sky130hd 平台下的 jpeg 完整的后端 flow，输出在 report, log, results, objects 这四个文件夹里面，你可以根据需要读取他们
 - (d) 最终给我一个完整的实验脚本并告诉我运行的方式。实验脚本直接放在 scripts 目录下，启动方式在对话里面写给我
4. 帮我修改一下这个脚本，让它能够接受如下输入字段：
 - (a) platform，为需要跑 clk_period 扫描的设计使用的 platform
 - (b) design，为需要跑 clk_period 扫描的设计的 nick name
 - (c) min，为 clk_period 扫描的最小值，单位是 ns
 - (d) max，为 clk_period 扫描的最大值，单位是 ns
 - (e) step，为 clk_period 扫描的步长，单位是 ns
 - (f) work_home，为输出的根目录，按照当前代码的默认值就是 tmp 这个文件夹

A.2 基于 OpenROAD 输出结果的统计工作

1. 请你查看 lab0_guide.pdf 这个文件的 Questions 这个章节，其中的问题 1. 2. 3. 4. 5. 需要的 flow 我已经跑完了，其中 5ns 的 sky130hd-jpeg 的后端 flow 运行结果在 OpenROAD-flow-scripts /flow/ 中的 results, log, reports, objects 这四个文件夹下面，而 clk_period 扫描重跑 OpenSTA 的结果在 tmp 文件夹下面。请你根据这些内容帮我写一份 lab0 的报告，要求用 latex 完成，主文件名名为 lab0_report.tex。本次任务中完成如下内容：
 - (a) 回答问题 1，按照 logs 文件夹中的命名来划分各个阶段，形成表格。注意表格中要区分主次两级阶段，并给每个次阶段分别写一句话解释这个阶段主要的作用是什么
 - (b) 回答问题 2，可以复用问题 1 的表格结构

- (c) 回答问题 3，这个问题题干里面没有要求具体内容，你需要在这个问题中参考 `OpenROAD-flow-scripts/flow/scripts/sta_clock_period_sweep.py` 这个实验脚本的内容，解释实验脚本需要完成什么（读取 baseline 的什么结果作为输入，修改什么参数，然后对哪个阶段重跑 OpenSTA），并展示最后的实验结果。在这个问题的回答中加上实验脚本的完整内容的跳转链接，完整脚本附在报告的最后
- (d) 回答问题 4。这个问题的答案可以从 log 里面抽取，但我还没有仔细检查过，如果你搜索之后发现 logs 里面没有对应内容请勿凭空作答，明确写出此部分缺乏数据
- (e) 回答问题 5，这部分所需的内容同样在 logs 中。你需要按照表格的形式组织答案，第一列标注这个 ERROR 发生在哪个主次阶段，第二列展示 ERROR 的报错原因，第三列回答是否可以 waive
2. 你的时序实例和组合实例的数量是怎么的出来的，是查看的 `OpenROAD-flow-scripts/flow/reports/sky130hd/jpeg/base/synth_stat.txt` 这个文件吗，时序实例和组合逻辑分别是哪些项目的 count 之和？请简要回答

B 完整实验脚本

以下收录了时钟频率扫描的完整实验脚本 `OpenROAD-flow-scripts/flow/scripts/sta_clock_period_sweep.py`，共 244 行，该脚本为作者与 gpt-6-Astra 对话生成，`effort="high"`。 [点击返回问题 3 的实验说明与结果。](#)

```

1  #!/usr/bin/env python3
2  """Sweep clock periods on an existing post-routing design (base variant).
3
4  Run from flow/: python3 scripts/sta_clock_period_sweep.py
5  Only Python's standard library, Make, and the installed OpenROAD are needed.
6  Use --help for platform, design, range, and output-directory options.
7  The final SDC's I/O delays are held fixed. All primary/virtual clocks with
8  the same baseline period are swept together. Independent clock domains
9  with different periods and explicit waveforms require manual review.
10 """
11
12 import argparse
13 import csv
14 from decimal import Decimal, InvalidOperation
15 import json
16 import math
17 import os
18 from pathlib import Path
19 import re
20 import shutil
21 import subprocess
22 import sys
23 import tempfile
24
25
26 DRC_KEY = "detailedroute__route__drc_errors"
27 TCL = """# Generated by sta_clock_period_sweep.py; analysis only.
28 source $::env(SCRIPTS_DIR)/load.tcl
29 load_design 6_final.odb scan.sdc
30 # Match final_connect.tcl's post-CTS clock treatment.
31 set_propagated_clock [all_clocks]
32 read_spef $::env(RESULTS_DIR)/6_final.spef
33 # The SDC was read in native library units; report all times in ns.
34 set_cmd_units -time ns
35 report_units

```

```

36 report_clock_properties [all_clocks]
37 check_setup
38 report_worst_slack -max -digits 9
39 report_wns -max -digits 9
40 report_tns -max -digits 9
41 report_checks -path_delay max -format full_clock_expanded \
42     -group_path_count 5 -digits 6 > $::env(REPORTS_DIR)/setup.rpt
43 report_checks -path_delay min -format full_clock_expanded \
44     -group_path_count 5 -digits 6 > $::env(REPORTS_DIR)/hold.rpt
45 puts "STA_SWEEP_COMPLETE"
46 exit
47 ""
48
49
50 def scan_sdc(original, period):
51     """Change primary/virtual periods, converting ns to native SDC units."""
52     found = []
53     lines = []
54     seconds = format(Decimal(period) * Decimal("1e-9"), "f")
55     number = r"[0-9.eE+-]+"
56     for line in original.splitlines(keepends=True):
57         if re.match(r"\s*create_clock\s", line):
58             if "-waveform" in line:
59                 raise ValueError("Explicit clock waveform needs manual review before sweeping.")
60             match = re.search(rf"-period\s+({number})(?=\s|$)", line)
61             if match is None:
62                 raise ValueError("Expected a numeric -period in final write_sdc output.")
63             found.append(Decimal(match[1]))
64             line, count = re.subn(
65                 rf"(-period\s+){number}(?=\s|$)",
66                 rf"<1>[sta::time_sta_ui {seconds}]", line,
67             )
68             if count != 1:
69                 raise ValueError("Expected one numeric -period per create_clock command.")
70             lines.append(line)
71         if not found:
72             raise ValueError("No primary/virtual clocks found in 6_final.sdc.")
73         if any(not p.is_finite() or p <= 0 for p in found) or len(set(found)) != 1:
74             raise ValueError("Clocks must share one positive baseline period; review independent clock domains first.")
75     return "".join(lines)
76
77
78 def positive_decimal(value):
79     try:
80         number = Decimal(value)
81     except InvalidOperation:
82         raise argparse.ArgumentTypeError("Expected a positive finite number.")
83     if not number.is_finite() or number <= 0:
84         raise argparse.ArgumentTypeError("Expected a positive finite number.")
85     return number
86
87
88 def directory_name(value):
89     if not re.fullmatch(r"[A-Za-z0-9_][A-Za-z0-9_-.]*", value):
90         raise argparse.ArgumentTypeError("Expected a platform/design directory name, not a path.")
91     return value
92
93
94 def scan_periods(minimum, maximum, step):
95     """Include max only when it lies on the min + n * step grid."""
96     return [format(minimum + i * step, "f")
97             for i in range(int((maximum - minimum) // step) + 1)]
98
99
100 def parse_args():
101     parser = argparse.ArgumentParser(description=__doc__)
102     parser.add_argument("--platform", type=directory_name, default="sky130hd")
103     parser.add_argument("--design", type=directory_name, default="jpeg", help="Design nickname (

```

```

104     default: jpeg).")
105     parser.add_argument("--min", type=positive_decimal, default=Decimal("5.6"), help="Minimum period
in ns (default: 5.6).")
106     parser.add_argument("--max", type=positive_decimal, default=Decimal("5.7"), help="Maximum period
in ns, inclusive when on the step grid (default: 5.7).")
107     parser.add_argument("--step", type=positive_decimal, default=Decimal("0.1"), help="Period step in
ns (default: 0.1).")
108     parser.add_argument("--work_home", "--work-home", type=Path, default=Path("tmp"), help="Output
root, relative to flow/ or absolute (default: tmp).")
109     args = parser.parse_args()
110     if args.min > args.max:
111         parser.error("--min must be <= --max.")
112     return args
113
114 def metric(log, label):
115     match = re.search(rf"~{re.escape(label)}\s+([+0-9.eE+])\s*$", log, re.MULTILINE)
116     if match is None or not math.isfinite(float(match[1])):
117         raise ValueError(f"No finite '{label}' in STA log.")
118     return float(match[1])
119
120
121 def main():
122     args = parse_args()
123     periods = scan_periods(args.min, args.max, args.step)
124     flow = Path(__file__).resolve().parents[1]
125     if Path.cwd().resolve() != flow:
126         raise ValueError("Run this script from the flow directory.")
127     design_path = Path(args.platform) / args.design
128     config = flow / "designs" / design_path / "config.mk"
129     if not config.is_file():
130         raise FileNotFoundError(config)
131     source = flow / "results" / design_path / "base"
132     for name in ("6_final.odb", "6_final.sdc", "6_final.spef"):
133         if not (source / name).is_file():
134             raise FileNotFoundError(source / name)
135     original = (source / "6_final.sdc").read_text()
136     constraints = {p: scan_sdc(original, p) for p in periods}
137
138     installed = flow.parent / "tools/install/OpenROAD/bin/openroad"
139     executable = os.environ.get("OPENROAD_EXE") or (
140         str(installed) if installed.is_file() else shutil.which("openroad")
141     )
142     if not executable or not shutil.which(executable):
143         raise FileNotFoundError("OpenROAD not found. Run inside the ORFS container or set
OPENROAD_EXE.")
144     executable = str(Path(shutil.which(executable)).resolve())
145
146     drc_file = flow / "logs" / design_path / "base/5_2_route.json"
147     drc = json.loads(drc_file.read_text()).get(DRC_KEY) if drc_file.is_file() else None
148     output_root = args.work_home.expanduser().resolve()
149     output_root.mkdir(parents=True, exist_ok=True)
150     output = Path(tempfile.mkdtemp(prefix=f"clk_period_sweep_{args.platform}_{args.design}_", dir=
output_root))
151     print(f"Output: {output}", flush=True)
152     shutil.copy2(source / "6_final.sdc", output / "baseline.sdc")
153     if drc_file.is_file():
154         shutil.copy2(drc_file, output / "baseline_route_metrics.json")
155     (output / "sta.tcl").write_text(TCL)
156     (output / "experiment.json").write_text(json.dumps({
157         "platform": args.platform,
158         "design": args.design,
159         "min_ns": str(args.min),
160         "max_ns": str(args.max),
161         "step_ns": str(args.step),
162         "work_home": str(output_root),
163         "periods_ns": periods,
164         "input_results": str(source),
165         "openroad_executable": executable,

```

```

166         "io_delay_policy": "Keep final SDC I/O delays fixed; sweep primary/virtual clocks together,
preserving generated-clock relationships.",
167         "drc_source": str(drc_file),
168         "drc_metric": DRC_KEY,
169         "detailed_route_drc_count": drc,
170     }, indent=2) + "\n")
171
172     rows = []
173     for period in periods:
174         trial = output / f"period_{period}ns"
175         trial.mkdir()
176         (trial / "scan.sdc").write_text(constraints[period])
177         # Read existing large artifacts through links; no extraction or routing.
178         for name in ("6_final.odb", "6_final.spef"):
179             (trial / name).symlink_to(source / name)
180         if (source / "memories").is_dir():
181             (trial / "memories").symlink_to(source / "memories", target_is_directory=True)
182         command = [
183             "make", "--no-print-directory", "-f", str(flow / "Makefile"), "run",
184             f"FLOW_HOME={flow}",
185             f"DESIGN_CONFIG={config}", f"PLATFORM={args.platform}",
186             f"DESIGN_NICKNAME={args.design}", "FLOW_VARIANT=base",
187             f"WORK_HOME={trial}", f"RESULTS_DIR={trial}",
188             f"REPORTS_DIR={trial}", f"LOG_DIR={trial}",
189             f"OBJECTS_DIR={trial / 'objects'}", f"SDC_FILE={trial / 'scan.sdc'}",
190             f"RUN_SCRIPT={output / 'sta.tcl'}", "RUN_LOG_NAME_STEM=sta",
191             f"OPENROAD_EXE={executable}", "NUM_CORES=4",
192         ]
193         (trial / "command.json").write_text(json.dumps(command, indent=2) + "\n")
194         print(f"Running STA at {period} ns ...", flush=True)
195         # Run even Make from tmp so incidental tool outputs also stay there.
196         env = dict(os.environ, TMPDIR=str(trial))
197         with (trial / "make.log").open("w") as log:
198             result = subprocess.run(command, cwd=trial, env=env, stdout=log, stderr=subprocess.STDOUT)
199
200         if result.returncode:
201             raise RuntimeError(f"STA failed at {period} ns; see {trial / 'make.log'}")
202         log = (trial / "sta.log").read_text()
203         if "STA_SWEEP_COMPLETE" not in log:
204             raise RuntimeError(f"STA did not complete; see {trial / 'sta.log'}")
205         slack = metric(log, "worst slack max")
206         row = {
207             "clk_period_ns": period,
208             "frequency_mhz": 1000 / float(period),
209             "worst_setup_slack_ns": slack,
210             "wns_ns": metric(log, "wns max"),
211             "tns_ns": metric(log, "tns max"),
212             "setup_pass": slack >= 0,
213             "estimated_limiting_period_ns": float(period) - slack,
214             "detailed_route_drc_count": drc if drc is not None else "unavailable",
215         }
216         rows.append(row)
217         with (output / "summary.csv").open("w", newline="") as stream:
218             writer = csv.DictWriter(stream, fieldnames=list(row))
219             writer.writeheader()
220             writer.writerows(rows)
221         print(f"  worst setup slack = {slack:.9f} ns; setup {'PASS' if slack >= 0 else 'FAIL'}",
flush=True)
222
223     passing = [row for row in rows if row["setup_pass"]]
224     notes = [
225         f"Fixed post-routing {args.platform}/{args.design}/base; OpenSTA analysis only.",
226         "Primary/virtual clocks swept together; all other final SDC constraints, including I/O delays
, fixed.",
227         f"Detailed-routing DRC count: {drc if drc is not None else 'unavailable'} (not a separate
signoff DRC check).",
228     ]
229     if passing:
230         best = min(passing, key=lambda row: float(row["clk_period_ns"]))

```

```

230     notes.append(f"Minimum passing TESTED period: {best['clk_period_ns']} ns; frequency: {best['
frequency_mhz']:.6f} MHz.")
231 else:
232     notes.append("No tested period passes setup timing.")
233 notes.append(f"Tested {len( periods )} periods from { periods[0]} to { periods[-1]} ns, step {args.
step} ns. The precise timing boundary may require a finer sweep.")
234 (output / "summary.txt").write_text("\n".join( notes ) + "\n")
235 print("\n".join( notes ))
236 print(f"Summary: {output / 'summary.csv'}")
237
238
239 if __name__ == "__main__":
240     try:
241         main()
242     except (OSError, ValueError, RuntimeError) as error:
243         print(f"ERROR: {error}", file=sys.stderr)
244         sys.exit(1)

```